



Παράλληλος και Κατανεμημένος Υπολογισμός

Εισαγωγή

Διεργασίες, ρουτίνες και νήματα

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2025



Εισαγωγή



Γιατί αυτό το μάθημα; Επειδή οι σημερινοί υπολογιστές ..

- Έχουν πολλούς επεξεργαστές που μας επιτρέπουν να επιταχύνουμε την εκτέλεση απαιτητικών εργασιών με κατάλληλο προγραμματισμό.
- Είναι συνήθως συνδεδεμένοι στο Διαδίκτυο, οι εφαρμογές και τα δεδομένα κατανέμονται σε υπολογιστές που συνεργάζονται.
- Εκτελούν πολλές εφαρμογές ταυτόχρονα, αλλά και κάθε εφαρμογή αποτελείται από επιμέρους εργασίες που εκτελούνται ταυτόχρονα.



Κεντρικό θέμα του μαθήματος: Περισσότεροι από ένας υπολογισμοί που ..

- **Εκτελούνται ταυτόχρονα:** δηλαδή πριν τελειώσει ένας υπολογισμός ξεκινάει κάποιος άλλος. Μπορεί να συμβαίνουν φυσικά ταυτόχρονα, εκτελούμενοι σε διαφορετικούς επεξεργαστές / υπολογιστές ή λογικά ταυτόχρονα, σε έναν επεξεργαστή, μέσω δρομολόγησης.
- **Ελέγχονται άμεσα** από το εκτελούμενο πρόγραμμα: δηλαδή είναι εμφανείς στον προγραμματιστή, δεν συμβαίνουν διαφανώς σε επίπεδο αρχιτεκτονικής ή λειτουργικού συστήματος αλλά συμβαίνουν επειδή έχουμε εισάγει τις κατάλληλες εντολές μέσα στο πρόγραμμά μας.

Ταυτόχρονος υπολογισμός

Concurrent computing

- Υπολογισμός όπου πολλαπλές εργασίες (tasks) εκτελούνται σε επικαλυπτόμενες χρονικές περιόδους.
- Στην ακολουθιακή εκτέλεση ένας υπολογισμός ξεκινά αφού ο προηγούμενος έχει τελειώσει, ενώ στη ταυτόχρονη εκτέλεση ένας υπολογισμός μπορεί να προχωρήσει χωρίς απαραίτητα να περιμένει την ολοκλήρωση άλλων υπολογισμών.
- Παράδειγμα: Εξυπηρέτηση πολλών αιτημάτων σε έναν διακομιστή. Ταυτόχρονες συναλλαγές (βάσεις δεδομένων). Διαχείριση πολλών παραθύρων σε μια εφαρμογή. Λειτουργικά συστήματα.

Κατανεμημένος υπολογισμός

Distributed computing

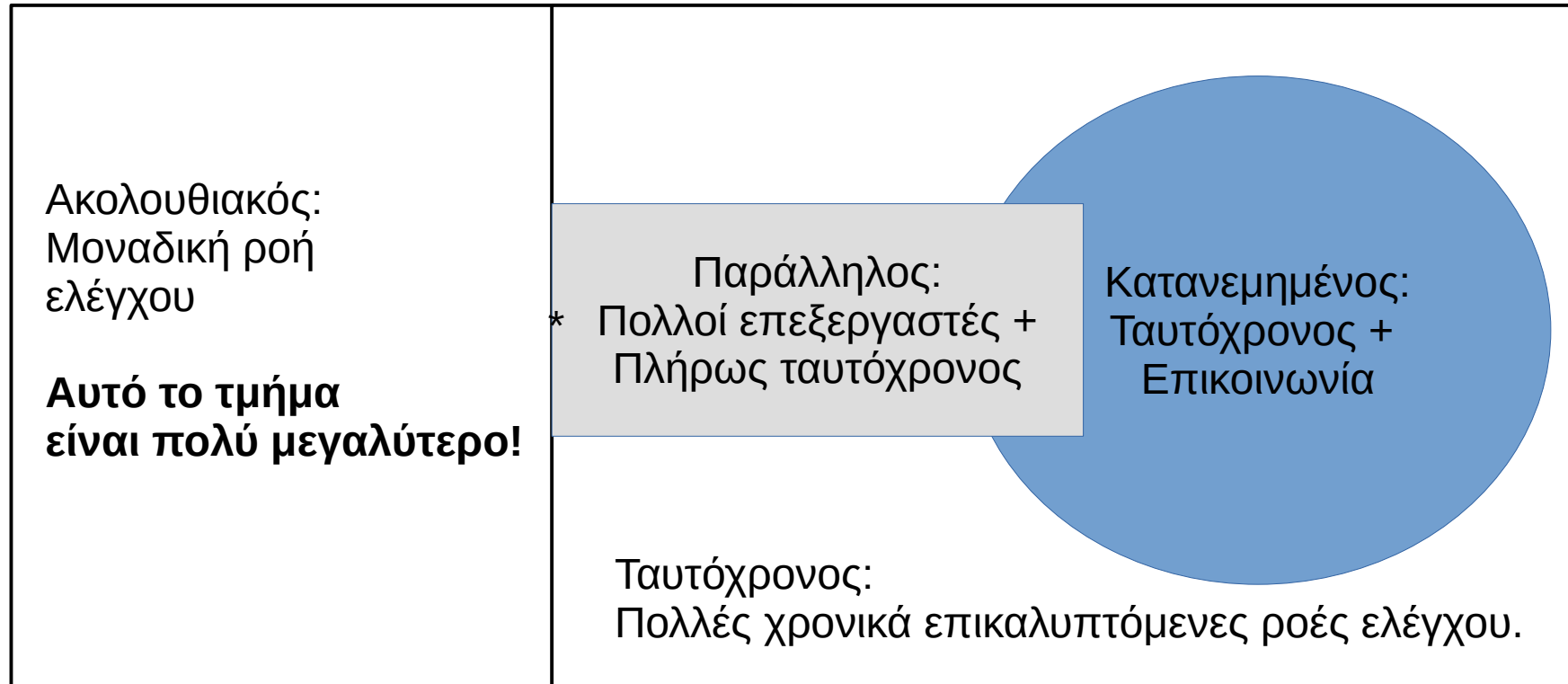
- Υπολογισμός που αποτελείται από εργασίες που εκτελούνται ταυτόχρονα σε ένα σύνολο δικτυωμένων υπολογιστών, είτε σε τοπικό δίκτυο (πχ κέντρα δεδομένων) ή απομακρυσμένους (πχ Διαδίκτυο), ώστε να φαίνεται στο χρήστη ότι αποτελούν μια ενιαία εφαρμογή.
- Παράδειγμα: Εφαρμογές Διαδικτύου. Εφαρμογές Παγκόσμιου Ιστού. Υπολογιστικό νέφος. Κινητές εφαρμογές. Αφανής και Διάχυτος Υπολογισμός.

Παράλληλος υπολογισμός

Parallel computing

- Υπολογισμός που αποτελείται από εργασίες που εκτελούνται ταυτόχρονα με τη χρήση υπολογιστικών συστημάτων τα οποία περιλαμβάνουν πολλούς δικτυωμένους υπολογιστές ή / και υπολογιστές που διαθέτουν πολλαπλές CPUs ή και πολλαπλές ALUs με στόχο τη ταχύτερη επίλυση ενός προβλήματος.
- Παράδειγμα: Γραφικά υπολογιστών. Επεξεργασία δεδομένων μεγάλου όγκου. Μηχανική μάθηση, Τεχνητά νευρωνικά δίκτυα. Υπολογισμοί σε κέντρα δεδομένων (πχ μηχανές αναζήτησης). Υπολογισμοί υψηλών επιδόσεων (συστοιχίες υπολογιστών).

Ακολουθιακός, ταυτόχρονος, κατανεμημένος, παράλληλος υπολογισμός



Υπολογισμός

Κάθε επιμέρους υπολογισμός ενός ταυτόχρονου, κατανεμημένου, παράλληλου υπολογισμού έχει τη δική του μοναδική ροή ελέγχου*.

*Μπορεί να έχουμε παράλληλο υπολογισμό με μια ροή ελέγχου αλλά πολλές ροές εκτέλεσης όταν έχουμε πολλές αριθμητικές λογικές μονάδες.

Σύνοψη διαγράμματος

- Ταυτόχρονος υπολογισμός: υπολογισμός με πολλαπλές ροές ελέγχου.
- Κατανεμημένος υπολογισμός: ταυτόχρονος υπολογισμός που εκτελείται σε δικτυωμένους υπολογιστές.
- Παράλληλος υπολογισμός: ταυτόχρονος υπολογισμός που εκτελείται σε πολλούς επεξεργαστές με κοινή μνήμη ή πολλούς δικτυωμένους υπολογιστές με στόχο τη βελτίωση της επίδοσης.*

* Υπάρχει μια κατηγορία παράλληλου υπολογισμού που έχει μόνο μια ροή ελέγχου και εκτελεί ταυτόχρονα την ίδια εντολή σε πολλές αριθμητικές λογικές μονάδες.

Ροή ελέγχου και συνεργασία

- Μια ροή ελέγχου είναι ένα τμήμα κώδικα (πχ μια ρουτίνα) που μπορεί να εκτελεστεί αυτόνομα. Συνήθως υλοποιείται ως διεργασία ή νήμα.

Πολλαπλές ροές ελέγχου: είτε πολλές διεργασίες ή πολλά νήματα σε μια διεργασία ή συνδυασμός.

- Η συνεργασία συνήθως υλοποιείται :
 - (α) επικοινωνία με κοινόχρηστες θέσεις μνήμης (μοιραζόμενες μεταβλητές συνήθως σημαίνει κοινή φυσική μνήμη).
 - (β) επικοινωνία με μεταβίβαση μηνυμάτων (συνήθως μέσω δικτύου αλλά μπορεί και σε ένα σύστημα).

Κοινή αφετηρία...

- Ο ταυτόχρονος, κατανεμημένος και παράλληλος υπολογισμός έχουν κοινό υπόβαθρο: πολλαπλές ροές ελέγχου που συνεργάζονται με κάποιο τρόπο. Έχουν (εν μέρει) κοινή θεωρία, ορολογία και τεχνικές προγραμματισμού.
- Ο ταυτόχρονος υπολογισμός, ως πιο θεμελιώδης, προσφέρει μια ικανοποιητική λογική αφαίρεση για τη μελέτη των βασικών αρχών (αλλά όχι του συνόλου) του παράλληλου και του κατανεμημένου υπολογισμού.

...αλλά διαφορετικοί στόχοι...

- Ο ταυτόχρονος υπολογισμός δίνει έμφαση στη συνεργασία εργασιών που εκτελούνται σε επικαλυπτόμενους χρόνους και μοιράζονται δεδομένα και πόρους.
- Ο κατανεμημένος υπολογισμός δίνει έμφαση στην δικτυακή επικοινωνία, στη συνεργασία εργασιών που εκτελούνται αυτόνομα σε πολλούς υπολογιστές.
- Ο παράλληλος υπολογισμός δίνει έμφαση στη ταχύτητα εκτέλεσης μέσω πολλών όμοιων εργασιών που εκτελούνται σε πολλούς επεξεργαστές ή αριθμητικές λογικές μονάδες.

...και ασαφή όρια

Παραδείγματα:

- Ένας ταυτόχρονος υπολογισμός, αν εκτελεστεί σε υπολογιστή με πολλούς επεξεργαστές μπορεί να γίνει, τουλάχιστο εν μέρει, παράλληλος.
- Ένας παράλληλος υπολογισμός μπορεί να περιλαμβάνει και επικοινωνία μεταξύ υπολογιστών, για παράδειγμα σε συστοιχίες ή κέντρα δεδομένων, άρα είναι και κατανεμημένος.
- Η υποδομή υπολογιστικού νέφους, των μηχανών αναζήτησης, των συστοιχιών υπολογιστών και τεχνητής νοημοσύνης είναι παράλληλη, κατανεμημένη και ταυτόχρονη..

Πολυεπίπεδη προσέγγιση

- Αλγόριθμοι: επίλυση προβλημάτων στο συγκεκριμένο τύπο υπολογισμού.
- Πρότυπα (patterns): εφαρμογή αλγοριθμικών και προγραμματιστικών προτύπων.
- Γλώσσες και τεχνικές προγραμματισμού: βιβλιοθήκες, δομές και τεχνικές προγραμματισμού.
- Υποστήριξη από το λειτουργικό σύστημα ή και ενδιάμεσο λογισμικό (middleware): διεργασίες, νήματα, επικοινωνία.
- Υποστήριξη από τα υπολογιστικά συστήματα: παράλληλες αρχιτεκτονικές και δίκτυα.

Τι θα κάνουμε

- Εισαγωγή στον παράλληλο, κατανεμημένο και ταυτόχρονο υπολογισμό, με έμφαση στις τεχνικές προγραμματισμού.
- Προγραμματισμός με νήματα / διεργασίες και με μοιραζόμενες μεταβλητές / μεταβίβαση μηνυμάτων.
- Η θεωρία και οι βασικές τεχνικές είναι σχετικά ανεξάρτητες από γλώσσα προγραμματισμού.
- Η υλοποίηση (προγραμματισμός) έχει διαφοροποιήσεις, ανάλογα με τη γλώσσα και βιβλιοθήκη επιλογής.

Επιθυμητές ιδιότητες:

Ασφάλεια, Ζωτικότητα, Επίδοση

Τα παράλληλα, κατανεμημένα, ταυτόχρονα προγράμματα ελέγχονται με βάση τις εξής ιδιότητες:

- Ασφάλεια (Safety): δεν συμβαίνει κάτι άσχημο κατά την εκτέλεση του προγράμματος, πχ κατάρρευση, αδιέξοδο, λάθος αποτελέσματα.
- Ζωτικότητα (Liveness): “τελικά” το πρόγραμμα έχει την επιθυμητή συμπεριφορά, παράγει το αναμενόμενο αποτέλεσμα.
- Επίδοση (Performance): το “τελικά” δεν περιλαμβάνει υπερβολικές, μη-αναγκαίες καθυστερήσεις κατά την εκτέλεση του προγράμματος.

Βιβλία και σημειώσεις

- Κεφάλαια από βιβλία που ήδη έχετε για τα μαθήματα Προγραμματισμός, Αλγόριθμοι, Αρχιτεκτονική, Λειτουργικά, Δίκτυα
- P. Pacheco, Εισαγωγή στο παράλληλο προγραμματισμό, εκδ. Κλειδάριθμος.
- M. Ben-Ari, Αρχές προγραμματισμού με ταυτοχρονισμό και κατανομή λογισμικού, εκδ. Τζιόλα.
- R. Bryant, D. O'Hallaron Συστήματα Υπολογιστών: μια προσέγγιση από τη πλευρά του προγραμματιστή, εκδ. Κλειδάριθμος.
- Σημειώσεις, κώδικας, σύνδεσμοι στο Open e-class

Εξετάσεις

- Παραδείγματα και ασκήσεις σχεδόν σε κάθε ενότητα.
- Οι εξετάσεις θα περιέχουν θεωρία και ύλη από τις ασκήσεις:
 - (α) Ερωτήσεις θεωρίας: συνήθως ερωτήσεις κρίσης
 - (β) Ζητήματα ανάλυσης και σχεδίασης: πώς σκεφτήκατε για τη λύση
 - (γ) Ζητήματα υλοποίησης: σκελετός κώδικα ή λεπτομερούς ψευδοκώδικα για τη λύση
- Έμφαση στον ουσιαστικό σχολιασμό: όχι απλή παράθεση κώδικα, όχι τετριμμένα σχόλια.
- Προσοχή στο άκριτο copy-paste, κρύβει παγίδες.

Εργασίες, εργαστήριο ..

- Οι μεγάλες εργασίες (με bonus) είναι η καλύτερη μέθοδος αλλά δεν δουλεύει λόγω εκτεταμένης αντιγραφής... και Chat-GPT.
- Το εργαστήριο (χωρίς bonus) δεν δουλεύει γιατί ελάχιστοι ασχολούνται κατά τη διάρκεια του εξαμήνου.

Πιθανή λύση ?

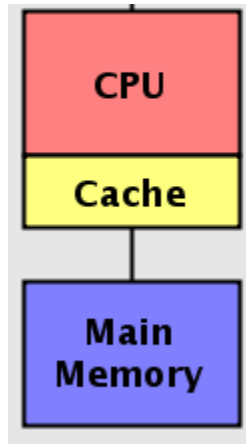


Διεργασίες, ρουτίνες και νήματα

Μοντέλο προγραμματισμού: πώς εκτελείται ένα πρόγραμμα;

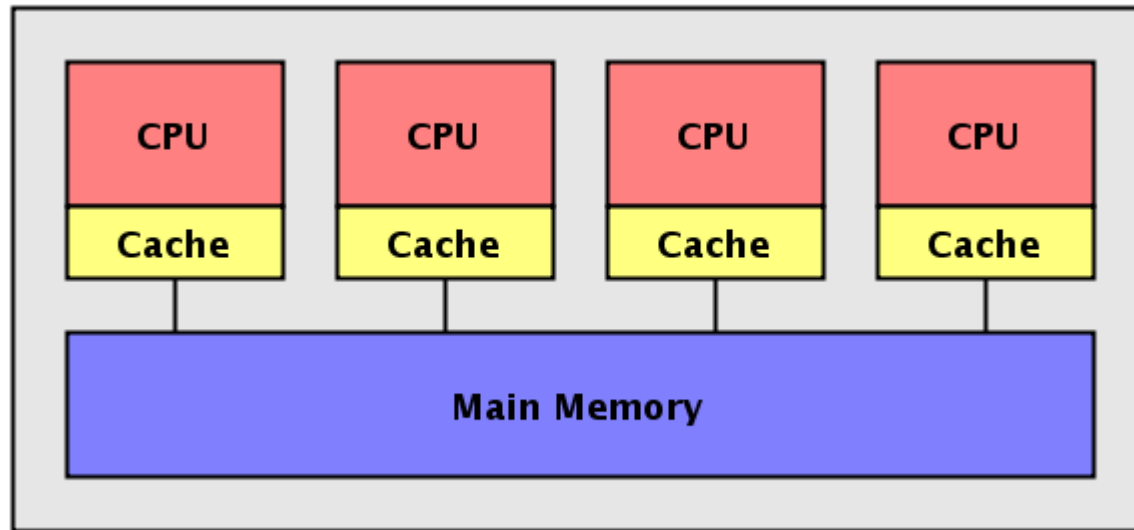
- Μοντέλο προγραμματισμού είναι η λογική αφαίρεση του υπολογιστικού συστήματος και του μηχανισμού εκτέλεσης προγραμμάτων που παρέχει η γλώσσα προγραμματισμού προς τον προγραμματιστή.
- Στον ακολουθιακό υπολογισμό η σημασία του μοντέλου προγραμματισμού είναι περιορισμένη λόγω της απλότητας και της γενικής εφαρμογής του μοντέλου.
- Δεν ισχύει το ίδιο για τον παράλληλο, κατανεμημένο και ταυτόχρονο υπολογισμό. Απαιτείται προσεκτικότερη μελέτη των συστημάτων και των μηχανισμών εκτέλεσης.

Μοντέλο υπολογιστικού συστήματος ακολουθιακού υπολογισμού



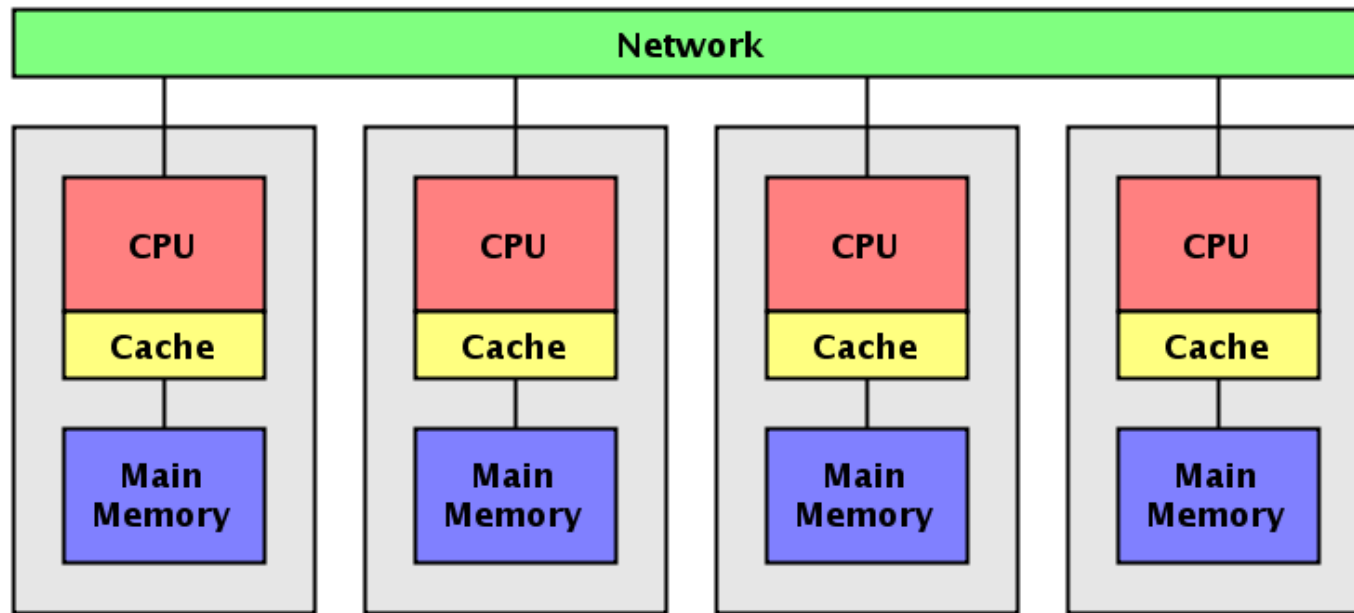
Ένας επεξεργαστής με κρυφή και κύρια μνήμη. Η κρυφή μνήμη μπορεί να έχει πολλά επίπεδα (L1,L2,L3). Η CPU μπορεί να εκτελεί πολλές εντολές ταυτόχρονα (Παραλληλισμός Επιπέδου Εντολής, ILP) αλλά ο προγραμματιστής “βλέπει” εκτέλεση μιας ακολουθίας εντολών (μια εντολή κάθε φορά). Επίσης μπορεί να διαθέτει πολλές ALUs ώστε να εκτελεί την ίδια εντολή σε πολλαπλά δεδομένα (Ανυσματική Επεξεργασία). Χρησιμοποιείται στον ακολουθιακό και στον ταυτόχρονο υπολογισμό.

Μοντέλο υπολογιστικού συστήματος μοιραζόμενης μνήμης



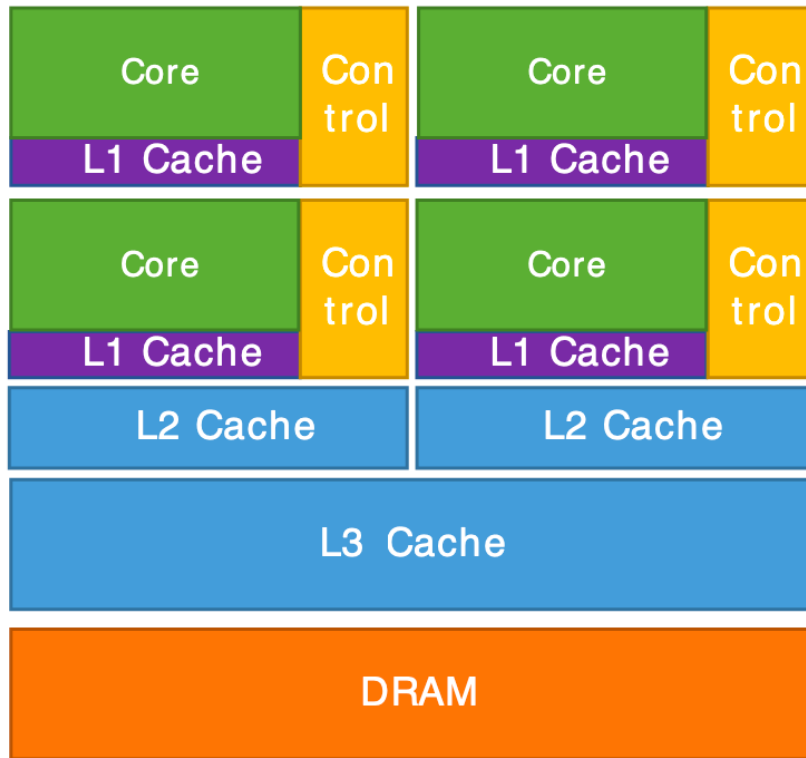
Πολλοί επεξεργαστές με ιδιωτική κρυφή μνήμη και κοινόχρηστη κύρια μνήμη. Τα ανώτερα τμήματα της κρυφής μνήμης μπορεί να είναι μοιραζόμενα, πχ L1, L2 ιδιωτικά και L3 μοιραζόμενη. Οι επεξεργαστές μπορεί να βρίσκονται στο ίδιο ή ξεχωριστά chip και έχουν τις ίδιες ιδιότητες με τον επεξεργαστή του μοντέλου ακολουθιακού υπολογισμού. Ο προγραμματιστής “βλέπει” να εκτελούνται ταυτόχρονα πολλές ακολουθίες εντολών, μια σε κάθε επεξεργαστή. Χρησιμοποιείται στον ταυτόχρονο και στον παράλληλο υπολογισμό.

Μοντέλο υπολογιστικού συστήματος κατανεμημένης μνήμης

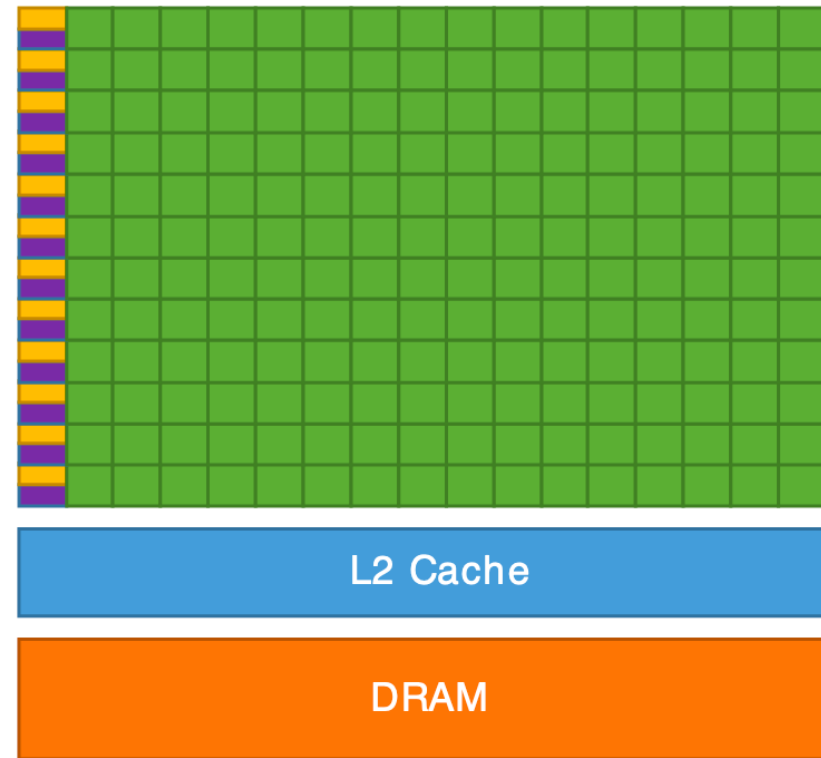


Ένα σύνολο από αυτόνομους υπολογιστές, όπως αυτοί του μοντέλου ακολουθιακού υπολογισμού, που επικοινωνούν μέσω δικτυακής υποδομής. Συνήθως το δίκτυο είναι τοπικό (πχ κέντρο δεδομένων, συστοιχία υπολογιστών) ή εκτεταμένο (να μεσολαβεί το Διαδίκτυο). Όμως υπάρχουν και εσωτερικά δίκτυα διασύνδεσης (σε έναν υπολογιστή, board, chip). Χρησιμοποιείται στον κατανεμημένο και στον παράλληλο υπολογισμό.

Άλλα μοντέλα υπολογιστικών συστημάτων



CPU



GPU

- Οι μονάδες γραφικών (GPUs) είναι ένα ακόμη συνηθισμένο μοντέλο, είτε αυτόνομο – είτε σε συνδυασμό με πολυπύρηνες CPUs.

Διεργασία 1

- Μια διεργασία είναι η λογική αφαίρεση ενός προγράμματος που εκτελείται. Το λειτουργικό σύστημα χειρίζεται διεργασίες.
- Τυπικά μια διεργασία έχει μια ροή ελέγχου, δηλαδή εκτελεί μια ακολουθία εντολών (ο εκτελέσιμος κώδικας).

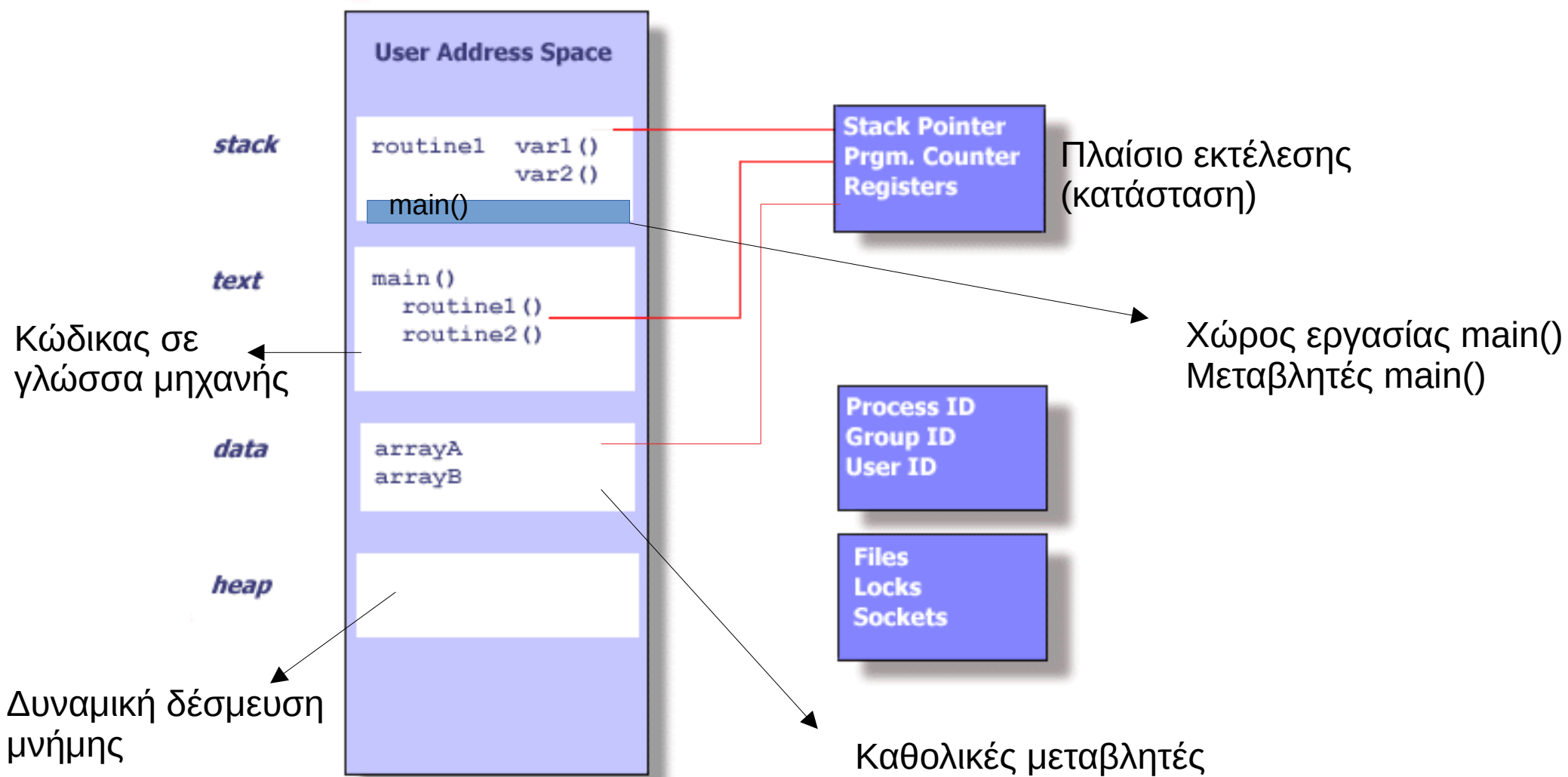
Το λειτουργικό σύστημα:

- παρέχει στη διεργασία χώρο στη μνήμη
- δρομολογεί τη διεργασία (χρόνος στη CPU) και
- παρακολουθεί την εκτέλεσή της (καταστάσεις).

Χώρος διευθύνσεων

- Χώρος διευθύνσεων λέγεται η λογική αφαίρεση για τον χώρο στη μνήμη που παρέχει το λειτουργικό σύστημα σε κάθε διεργασία. Μεταξύ άλλων περιλαμβάνει:
- Εκτελέσιμο κώδικα : το πρόγραμμα (text)
- Καθολικές μεταβλητές (data)
- Χώρο δυναμικής δέσμευσης μνήμης (heap)
- Στοίβα κλήσεων (stack)
- Ανοικτά αρχεία, κλήσεις συστήματος, σήματα ..
- Πλαίσιο εκτέλεσης

Διεργασία 2

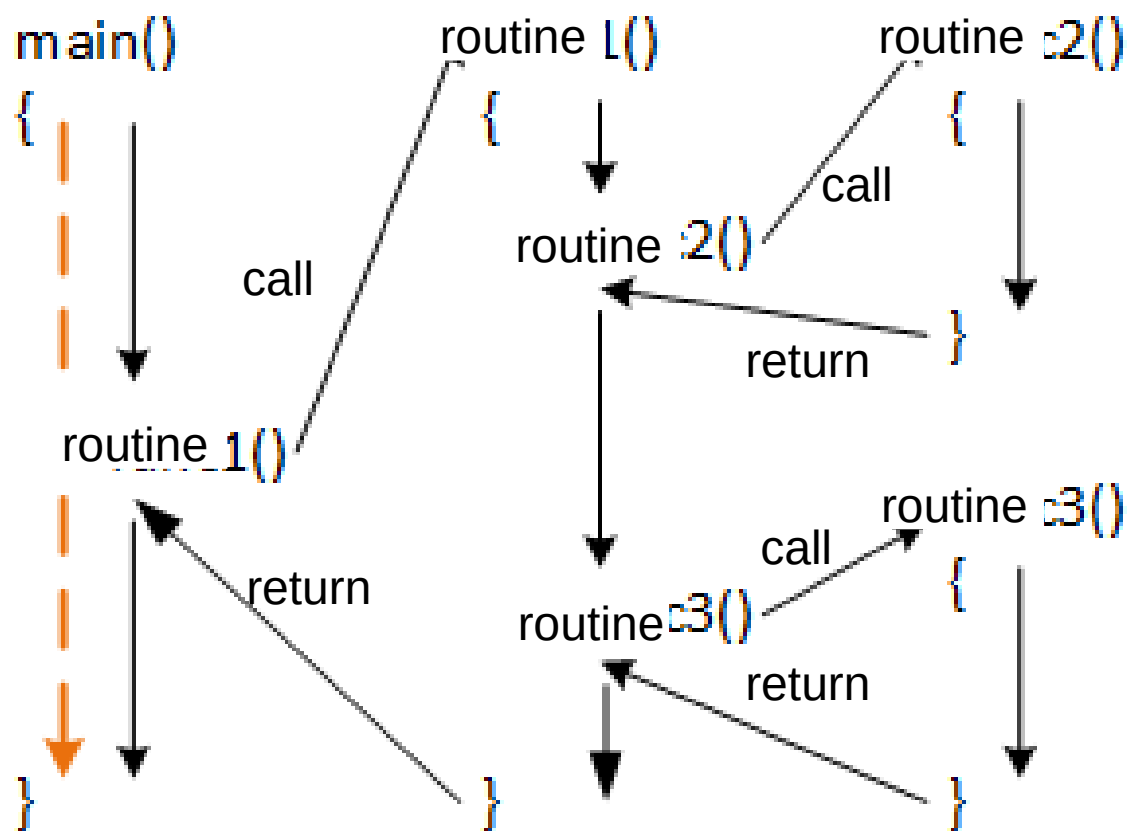


Χώρος διευθύνσεων και πλαίσιο εκτέλεσης μιας διεργασίας με μια ροή ελέγχου.

Ρουτίνα 1

- Ρουτίνα (διαδικασία, συνάρτηση ή μέθοδος) λέγεται ένα τμήμα του κώδικα το οποίο έχει μια λογική αυτονομία.
- Στις περισσότερες μορφές προγραμματισμού (διαδικαστικό, αντικειμενοστρεφή, συναρτησιακό κλπ) η ρουτίνα αποτελεί τη βάση δόμησης του προγράμματος.
- Το `main` είναι μια ρουτίνα, η οποία είναι η πρώτη που ξεκινά, και συνήθως καλεί άλλες ρουτίνες (Call). Η ροή εκτέλεσης του προγράμματος συνεχίζει στην ρουτίνα.
- Όταν η ρουτίνα τερματιστεί (Return) η ροή εκτέλεσης επιστρέφει στην εντολή αμέσως μετά την κλήση.
- Κάθε ρουτίνα μπορεί να καλέσει διαδοχικά άλλες ρουτίνες (ή και τον εαυτό της, αναδρομή) κ.ο.κ.

Ρουτίνα 2



Ροή εκτέλεσης



Στοίβα κλήσεων (στοίβα εκτέλεσης) 1

- Όταν καλείται μια ρουτίνα, στη στοίβα κλήσεων δημιουργείται ένας νέος χώρος εργασίας, που δείχνεται από τον Stack Pointer, και περιέχει τα ορίσματα και τις τοπικές μεταβλητές της ρουτίνας.
- Όταν η ρουτίνα τερματιστεί, αφού αποθηκευτούν οι τιμές επιστροφής, ο χώρος εργασίας ακυρώνεται.
- Διαδοχικές κλήσεις σημαίνουν διαδοχικούς χώρους εργασίας, ενεργός είναι ο πιο πρόσφατος χώρος εργασίας.
- Ο χώρος εργασίας του main βρίσκεται στην αρχή της στοίβας και συνήθως παραμένει ενεργός μέχρι τον τερματισμό του προγράμματος.

Στοίβα κλήσεων (στοίβα εκτέλεσης) 2

```
int[] arrayA[10];  
int[] arrayB[10];
```

```
routine1 () {  
    int var1;  
    int var2;  
    ...  
}
```

```
routine2 () {  
    int var1;  
    int var2;  
    int var3;  
    ...  
}
```

```
main() {  
    int a;  
    ...  
    routine1();  
    routine2();  
    ...  
}
```

Στοίβα εκτέλεσης: χρονική εξέλιξη

int a;

//int a;

int var1;
int var2;

//int a;

int var1;
int var2;
int var3;

int a;

Οι καθολικές μεταβλητές συνεχώς ορατές

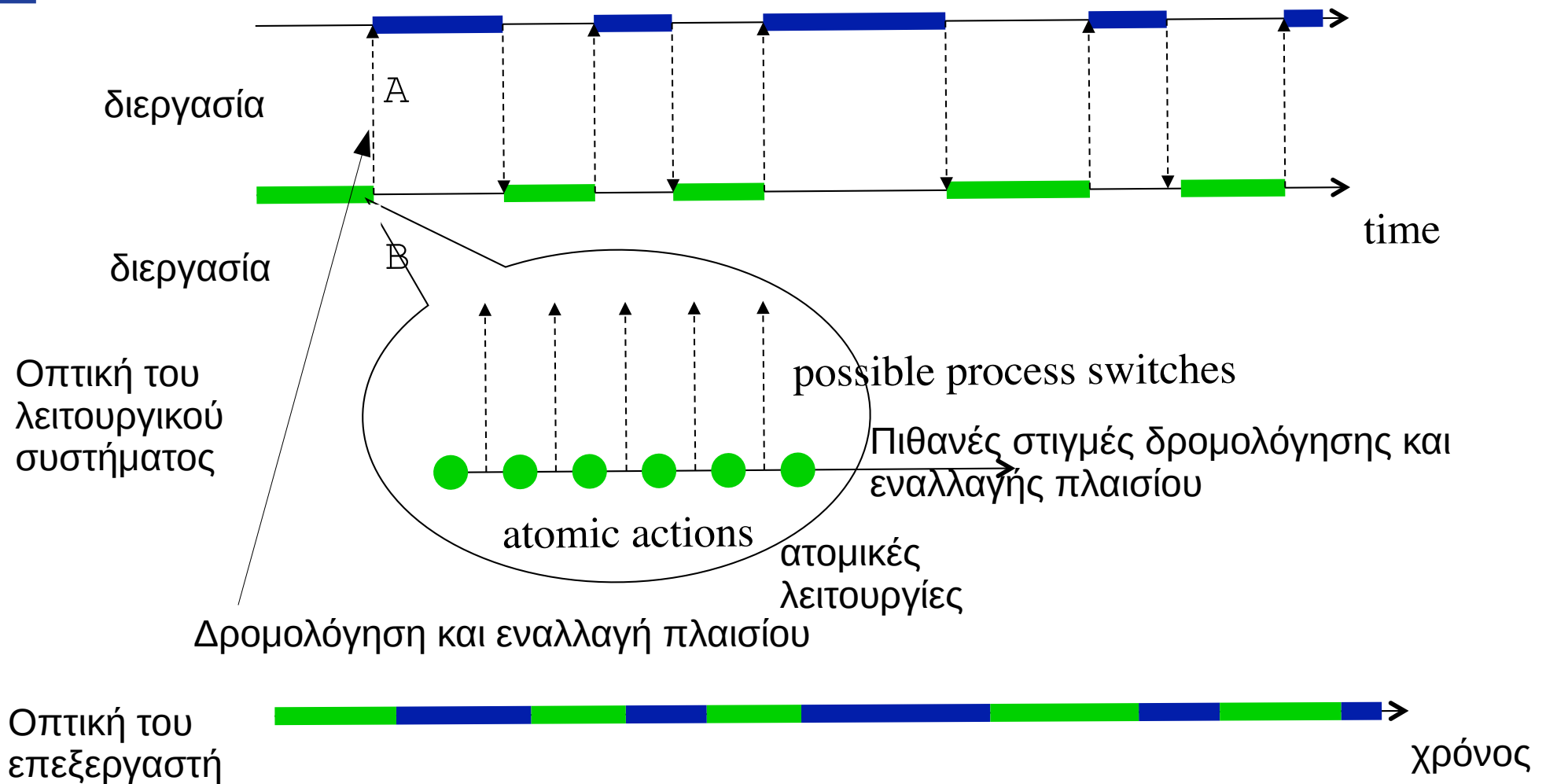
Πολλαπλές ταυτόχρονες διεργασίες

- Βασική λειτουργία των λειτουργικών συστημάτων.
- Το λειτουργικό σύστημα διαχειρίζεται τον χρόνο του επεξεργαστή (ή των επεξεργαστών).
- Χρονοδρομολογεί τις διεργασίες, ώστε αυτές εκτελούνται ταυτόχρονα.
- Κάθε φορά εκτελείται μια διεργασία σε κάθε επεξεργαστή. Κάθε διεργασία διαθέτει τη δική της ροή εκτέλεσης. Όταν η διεργασία διακοπεί για να εκτελεστεί κάποια άλλη διεργασία, το λειτουργικό σύστημα είναι υπεύθυνο να διατηρεί την κατάστασή της και να την επαναφέρει αργότερα (δρομολόγηση).

Πλαίσιο εκτέλεσης, δρομολόγηση

- Πλαίσιο εκτέλεσης λέγεται το σύνολο των τιμών των καταχωρητών του επεξεργαστή που απεικονίζουν την τρέχουσα κατάσταση της CPU και του χώρου διευθύνσεων μετά από την εκτέλεση κάθε εντολής.
- Ο κύκλος fetch - execute είναι ατομική λειτουργία, δε διακόπτεται. Τυχόν εξαιρέσεις, διακοπές κλπ ελέγχονται στο τέλος κάθε κύκλου fetch – execute.
- Επομένως η δρομολόγηση μπορεί να συμβεί στο τέλος του κύκλου fetch-execute μιας εντολής γλώσσας μηχανής.
- Στη δρομολόγηση διεργασίας συμβαίνει εναλλαγή του αντίστοιχου πλαισίου (context switch): αποθήκευση τρέχοντος πλαισίου και φόρτωση νέου.

Ατομική λειτουργία και δρομολόγηση



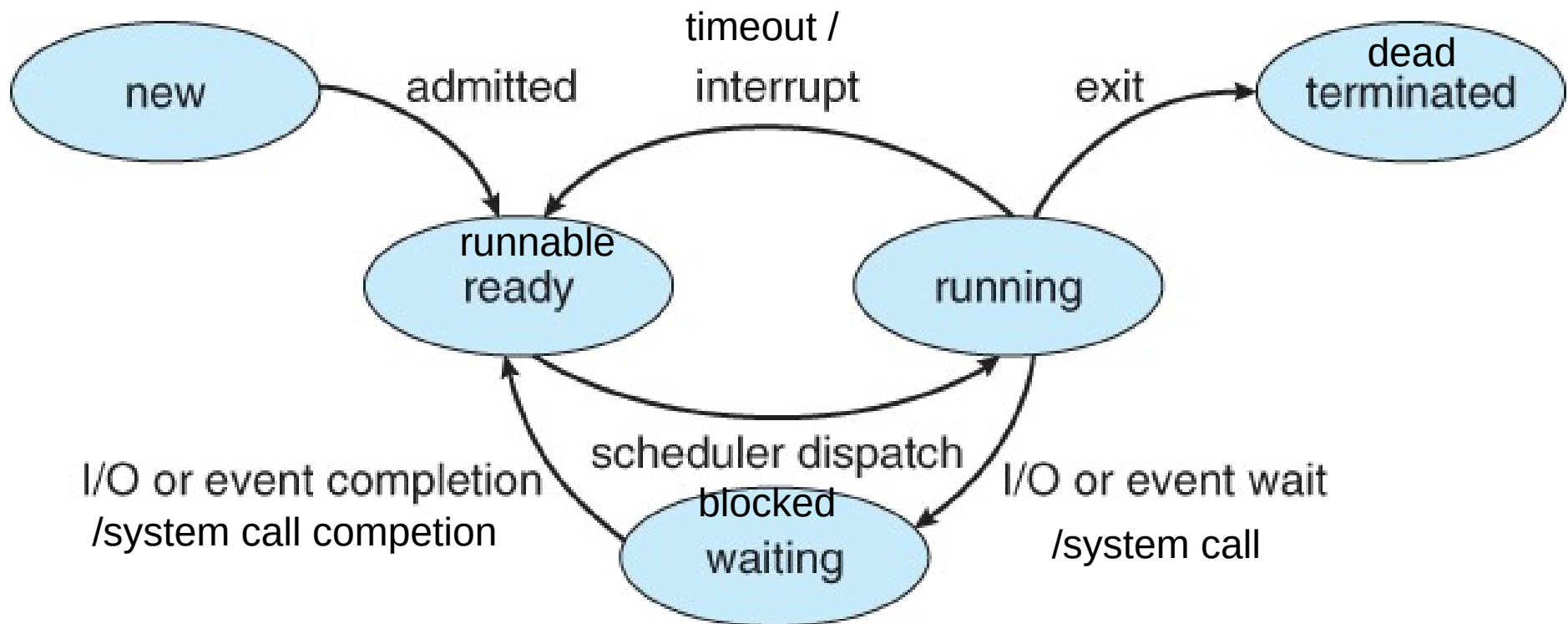
Το σχήμα είναι απλουστευτικό: δεν εμφανίζει τη συμμετοχή του λειτουργικού συστήματος σε κάθε δρομολόγηση και εναλλαγή πλαισίου, και υποθέτει ότι οι δύο διεργασίες καθίστανται έγκαιρα ξανά διαθέσιμες προς εκτέλεση.

Πότε συμβαίνει δρομολόγηση;

- Όταν μια διεργασία καταναλώσει το χρονομερίδιο εκτέλεσης (κβάντο) που της έχει εκχωρήσει το λειτουργικό σύστημα. Ισχύει για υπολογιστικά εντατικές (cpu-bound) διεργασίες.
- Όταν μια διεργασία εκτελέσει μια κλήση συστήματος (system call), δηλαδή προκαλέσει μια διακοπή λογισμικού για να ζητήσει κάτι από το λειτουργικό σύστημα, συνήθως I/O. Ισχύει για διαλογικές (I/O bound) διεργασίες.
- Όταν υπάρχει κάποια εξαίρεση (σφάλμα εκτέλεσης) ή διακοπή υλικού.

Δρομολόγηση και καταστάσεις διεργασίας

Το λειτουργικό σύστημα παρακολουθεί τη κατάσταση της διεργασίας.



Κάθε μετάβαση μεταξύ καταστάσεων σημαίνει δρομολόγηση.
Η μετάβαση γίνεται όταν συμβεί το αντίστοιχο γεγονός.

Δρομολόγηση και μη – αιτιοκρατική εκτέλεση σε έναν επεξεργαστή

- Οι διεργασίες έχουν περιορισμένο έλεγχο στη δρομολόγησή τους: η δρομολόγηση και οι κλήσεις συστήματος εκτελούνται από το λειτουργικό σύστημα.
- Προεκτόπιση (pre-emption) = δρομολόγηση πριν τη λήξη της εκτέλεσης. Οι διεργασίες εκτελούνται ουσιαστικά μη -αιτιοκρατικά.
- Δεν μπορούμε να είμαστε σίγουροι για τη σχετική ταχύτητα εκτέλεσής τους, δηλαδή σε ποια χρονική στιγμή (σχετική με την εκτέλεση μιας άλλης ταυτόχρονης διεργασίας) θα εκτελέσουν τη συγκεκριμένη εντολή ή θα διακόψουν την εκτέλεσή τους.

Δρομολόγηση και μη – αιτιοκρατική εκτέλεση σε σύστημα μοιραζόμενη μνήμης

- Ένα σύστημα μοιραζόμενης μνήμης επιτρέπει την πλήρως ταυτόχρονη (παράλληλη) εκτέλεση πολλών διεργασιών.
- Όμως το λειτουργικό παραμένει ένα: η σειρά εξυπηρέτησης των κλήσεων συστήματος δεν μπορεί να καθοριστεί. Το λειτουργικό σύστημα πρέπει να εκτελεστεί και αυτό σε κάποιο επεξεργαστή.
- Η ταυτόχρονη προσπάθεια στη μοιραζόμενη μνήμη από τους επεξεργαστές αναγκαστικά σειριοποιεί μη-προσδιοριστικά τις αναγνώσεις – εγγραφές που εκτελούν οι διεργασίες.

Δρομολόγηση χωρίς προεκτόπιση*

- Σε ορισμένα συστήματα η προεκτόπιση δεν είναι επιθυμητή. Προσπαθούμε να επιτύχουμε προσδιοριστική εκτέλεση εφαρμογών, προβλέψιμη συμπεριφορά.
- Σε αυτές τις περιπτώσεις δεν ισχύει η δρομολόγηση λόγω χρονοδιακοπής αλλά μόνο λόγω αναστολής εκτέλεσης για I/O ή άλλης κλήσης συστήματος, εξαίρεσης ή εξωτερικής διακοπής.

Νήματα 1: πολλαπλές ταυτόχρονες ροές εκτέλεσης μέσα σε μια διεργασία

- Τα νήματα είναι μηχανισμός που επιτρέπει την ύπαρξη πολλαπλών ταυτόχρονων ροών εκτέλεσης μέσα σε μια διεργασία.
- Τα νήματα αποτελούν τμήμα μιας διεργασίας, δεν είναι ανεξάρτητα.
- Μια απλή διεργασία (με μια ροή εκτέλεσης) έχει ένα νήμα.

Αρχικά χρησιμοποιήθηκαν για:

- Καλύτερη διαχείριση μνήμης και κώδικα.
- Μείωση του αριθμού των θεματικών εναλλαγών μιας διεργασίας.

Νήματα 2: καλύτερη διαχείριση μνήμης και κώδικα

- Πολλαπλές κλήσεις της ίδιας εφαρμογής (πχ ενός περιηγητή ή ενός διακομιστή ιστού) δεν απαιτούν τη δημιουργία πολλαπλών διεργασιών. Μια διεργασία έχει πολλά νήματα, τα οποία εκτελούν τον ίδιο κώδικα αλλά σε διαφορετικά δεδομένα. Μείωση αριθμού διεργασιών και της απαιτούμενης μνήμης.
- Κάθε τμήμα κώδικα που εκτελεί διαφορετικό I/O ανατίθεται σε ξεχωριστό νήμα (πχ πληκτρολόγιο, οθόνη, δίσκος, δικτυακή σύνδεση). Διευκόλυνση στη διαχείριση I/O. Παράδειγμα: πελάτης chat.

Νήματα 3: μείωση θεματικών εναλλαγών της διεργασίας

- Οι διαλογικές (I/O bound) διεργασίες εκτελούν πολύ συχνές κλήσεις συστήματος (system calls) με αποτέλεσμα την αναστολή εκτέλεσής τους, θεματική εναλλαγή και νέα δρομολόγηση.
- Υπερβολικός αριθμός δρομολογήσεων διεργασιών.
- Διαλογικές διεργασίες με νήματα: Το νήμα που εκτελεί κλήση συστήματος αναστέλλεται, αλλά άλλα νήματα της ίδιας διεργασίας μπορούν να συνεχίσουν να εκτελούνται (μέχρι και αυτά να εκτελέσουν κλήση συστήματος).
- Μείωση αριθμού δρομολογήσεων διεργασιών.

Νήματα 4: παράλληλος υπολογισμός

Τα νήματα έγιναν ιδιαίτερα δημοφιλή μετά την ανάπτυξη επεξεργαστών που χρησιμοποιούν τις τεχνολογίες:

- Ταυτόχρονη πολυνημάτωση (simultaneous multithreading): επεξεργαστές που μπορούν να εκτελούν εντολές από διαφορετικά νήματα ταυτόχρονα (συνήθως δύο νήματα).
- Πολυπύρρηνοι επεξεργαστές (multicore CPUs): επεξεργαστές που περιέχουν 2, 4, 8, 16.. πλήρεις επεξεργαστές με ιδιωτική κρυφή μνήμη.

Τα νήματα επικράτησαν ως τρόπος παράλληλου προγραμματισμού.

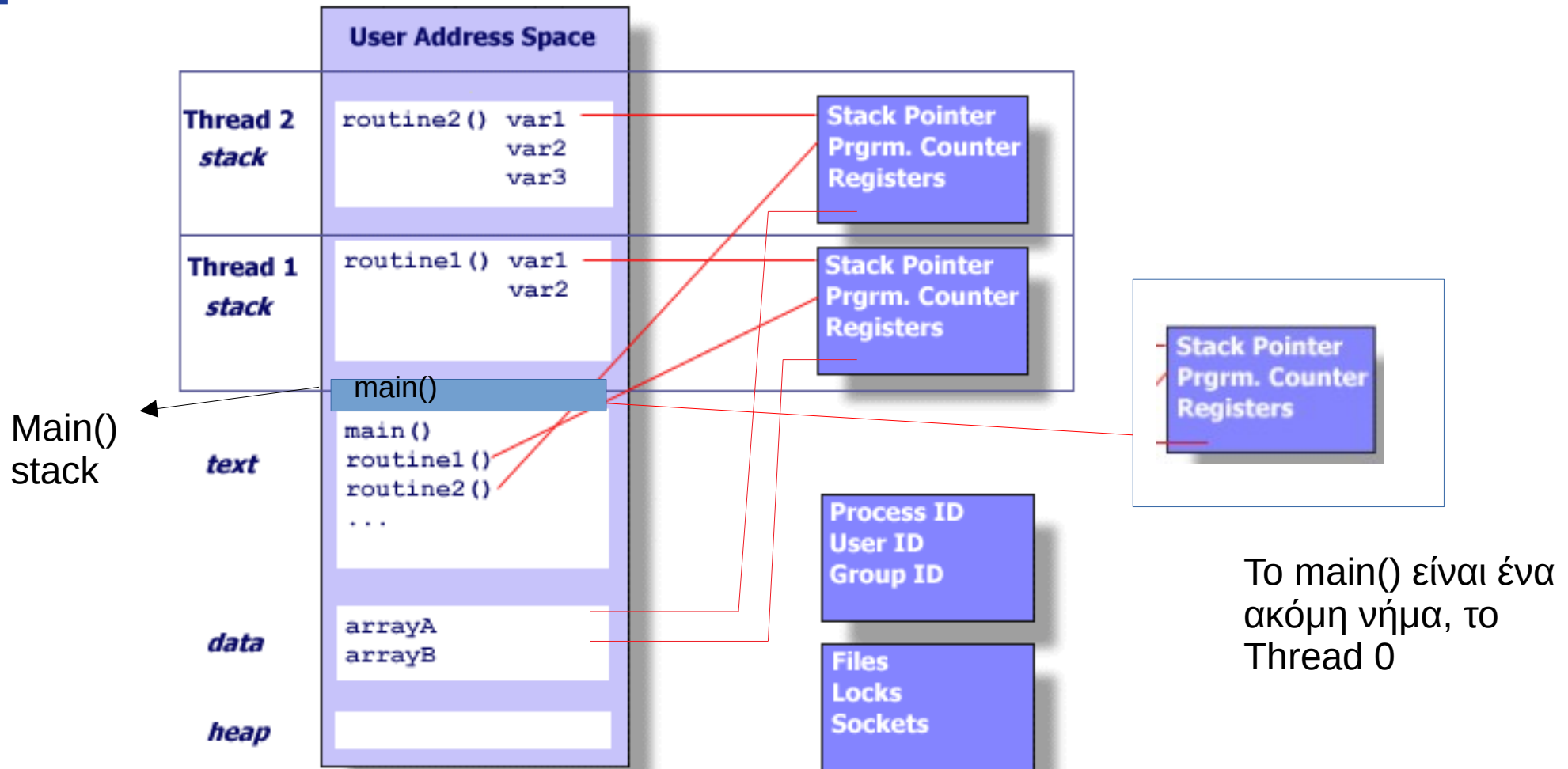
Νήματα και παραλληλισμός επιπέδου εντολής 1

- Παραλληλισμός Επιπέδου Εντολής (ILP): οι τεχνικές που χρησιμοποιεί η αρχιτεκτονική για την επιτάχυνση της εκτέλεσης των εντολών: διοχέτευση, πρόβλεψη διακλάδωσης, υπερβαθμωτή εκτέλεση, κρυφή μνήμη κλπ.
- Επιπλέον ορισμένοι επεξεργαστές έχουν τη δυνατότητα ταυτόχρονης εκτέλεσης δύο ή περισσότερων νημάτων σε έναν πυρήνα (core) με τεχνικές όπως το simultaneous multithreading, hyperthreading.
- Έτσι σε επίπεδο αρχιτεκτονικής μπορούμε να ορίσουμε `νήματα υλικού` ή `λογικούς πυρήνες`.

Νήματα και παραλληλισμός επιπέδου εντολής 2

- Γενικά τα θέματα αρχιτεκτονικής δεν μας απασχολούν εδώ.
- Εμείς ενδιαφερόμαστε για τον αριθμό CPUs που `βλέπει` το λειτουργικό σύστημα και δρομολογεί τις διεργασίες και τα νήματα. Δεν μας ενδιαφέρει με ποιο τρόπο είναι υλοποιημένες οι CPUs στο υλικό (πχ 2 πυρήνες με 2 νήματα υλικού ανά πυρήνα ή 4 πυρήνες με ένα νήμα υλικού ανά πυρήνα δίνουν και οι δύο 4 CPUs στο λειτουργικό σύστημα)
- Τα νήματα στα οποία αναφερόμαστε είναι καθαρά έννοια του λογισμικού, δηλαδή ο προγραμματιστής εφαρμογών καθορίζει τη δημιουργία, εκτέλεση, τερματισμό των νημάτων.

Διεργασία με νήματα 1



Χώρος διευθύνσεων και πλαίσια εκτέλεσης μιας διεργασίας με δύο επιπλέον ροές ελέγχου (δύο νήματα).

Στοίβες κλήσεων (στοίβες εκτέλεσης)

```
int[] arrayA[10];  
int[] arrayB[10];
```

```
routine1 () {  
    int var1;  
    int var2;  
    ...  
}
```

```
routine2 () {  
    int var1;  
    int var2;  
    int var3;  
    ...  
}
```

```
main() {  
    int a;  
    ...  
    create/start thread (routine1());  
    create/start thread (routine2());  
    ...  
}
```

Στοίβες εκτέλεσης: ανεξάρτητες μετά τη δημιουργία

int a;

...

int var1;
int var2;

...

int var1;
int var2;
int var3;

...

Καθολικές μεταβλητές ορατές σε
όλα τα νήματα

Διεργασία με νήματα 2

Τα νήματα έχουν **κοινά**:

- Εκτελέσιμο κώδικα (text)
- Καθολικές μεταβλητές στατικά δηλωμένες (data)
- Χώρο δυναμικής δέσμευσης μνήμης (heap)
- Ανοικτά αρχεία,

Τα νήματα έχουν **ξεχωριστά**:

- Program Counter.
- Στοίβα κλήσεων (stack)
- Πλαίσιο εκτέλεσης (περιορισμένο, δεν αλλάζει ο χώρος διευθύνσεων).
- Κλήσεις συστήματος, σήματα (ορισμένα σήματα αναφέρονται στη διεργασία)

Διεργασία με νήματα 3

- Κοινός κώδικας, ξεχωριστοί Program Counters:

Το κάθε νήμα μπορεί να εκτελεί άλλο τμήμα του προγράμματος

- Ξεχωριστές στοίβες κλήσεων

Το κάθε νήμα έχει ξεχωριστό χώρο εργασίας (τοπικές μεταβλητές και ορίσματα), και μπορεί να καλεί διαφορετικές συναρτήσεις

- Ξεχωριστές κλήσεις συστήματος, σήματα

Το κάθε νήμα μπορεί να περιμένει άλλη υπηρεσία από το λειτουργικό σύστημα, και να δέχεται άλλες ειδοποιήσεις (σήματα)

Καταστάσεις και δρομολόγηση νημάτων

Τα νήματα δρομολογούνται και έχουν παρόμοιες καταστάσεις με αυτές των διεργασιών:

- New: δημιουργία
- Ready(Runnable): έτοιμο προς εκτέλεση
- Running: εκτελείται, έχει δρομολογηθεί
- Blocked: σε αναστολή, αναμονή κάποιου γεγονότος, όπως κλήση συστήματος, σήμα, κάποια εξαίρεση ..
- Dead: τερματισμός

Οι συνθήκες δρομολόγησης και μετάβασης από κατάσταση σε κατάσταση είναι παρόμοιες με τις διεργασίες.

Δρομολόγηση νημάτων 1

Ισχύουν παρόμοιες συνθήκες όπως και στις διεργασίες, αλλά:

- Η δημιουργία, διαχείριση και δρομολόγηση νημάτων είναι πολύ ταχύτερη και οικονομικότερη (σε μνήμη) από ότι των διεργασιών.
- Τα νήματα έχουν περιορισμένο πλαίσιο εκτέλεσης και δεν απαιτείται αλλαγή χώρου διευθύνσεων. Γι' αυτό προτιμούμε (όταν μπορούμε) να έχουμε πολλά νήματα και όχι πολλές διεργασίες.

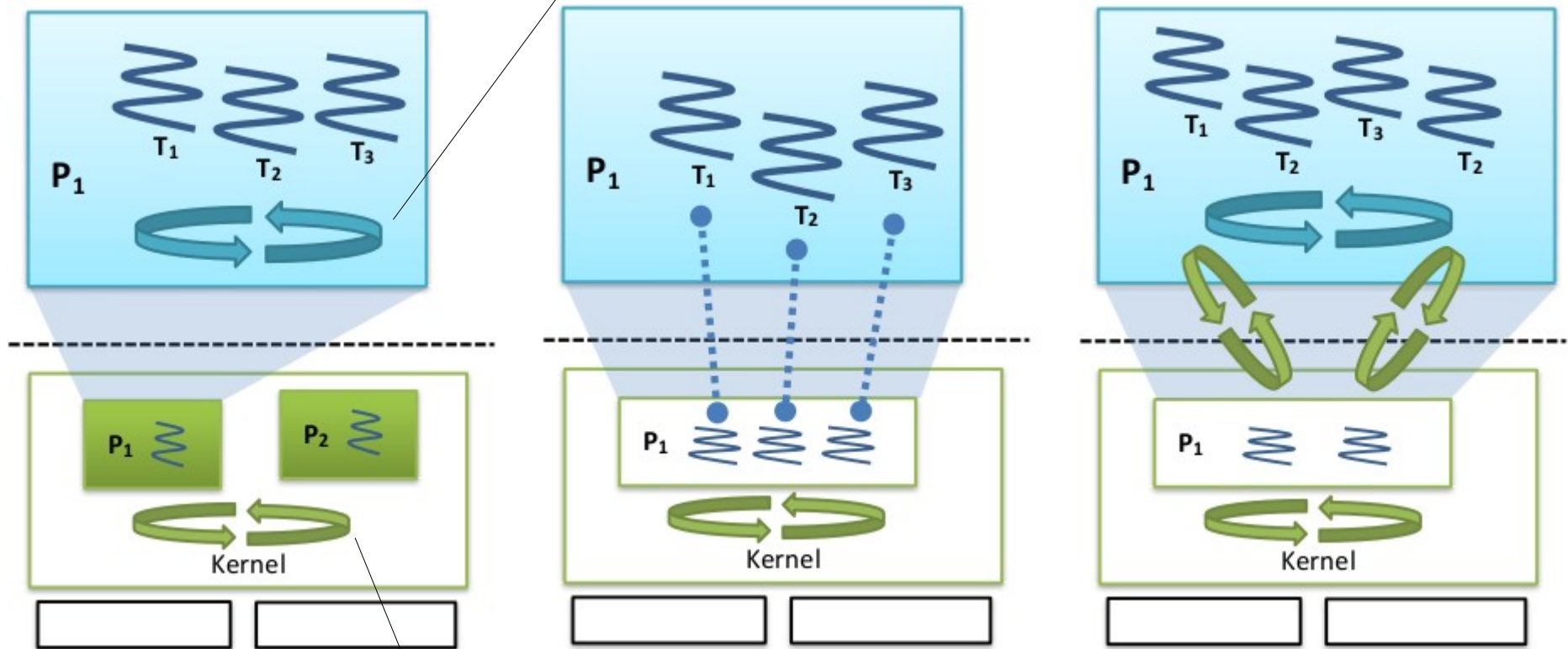
Δρομολόγηση νημάτων 2

Δύο επίπεδα δρομολόγησης

- Δρομολόγηση της διεργασίας που περιέχει τα νήματα. Ένα νήμα μπορεί να εκτελεστεί μόνο όταν εκτελείται η διεργασία που το περιέχει.
- Δρομολόγηση των νημάτων της διεργασίας. Η δημιουργία, διαχείριση και δρομολόγηση νημάτων μπορεί να υλοποιηθεί σε διαφορετικά επίπεδα:
 - Επίπεδο χρήστη (στο σύστημα χρόνου εκτέλεσης της διεργασίας)
 - Επίπεδο πυρήνα (στο λειτουργικό σύστημα)
 - Συνδυασμό των παραπάνω

Δρομολόγηση νημάτων 3

Δρομολόγηση από το σύστημα χρόνου εκτέλεσης της γλώσσας / εικονικής μηχανής



N:1
Ένα νήμα κάθε φορά

N:N
Όλα τα νήματα ορατά

N:M
M < N νήματα κάθε φορά

Δρομολόγηση από το λειτουργικό σύστημα

Δρομολόγηση N:1

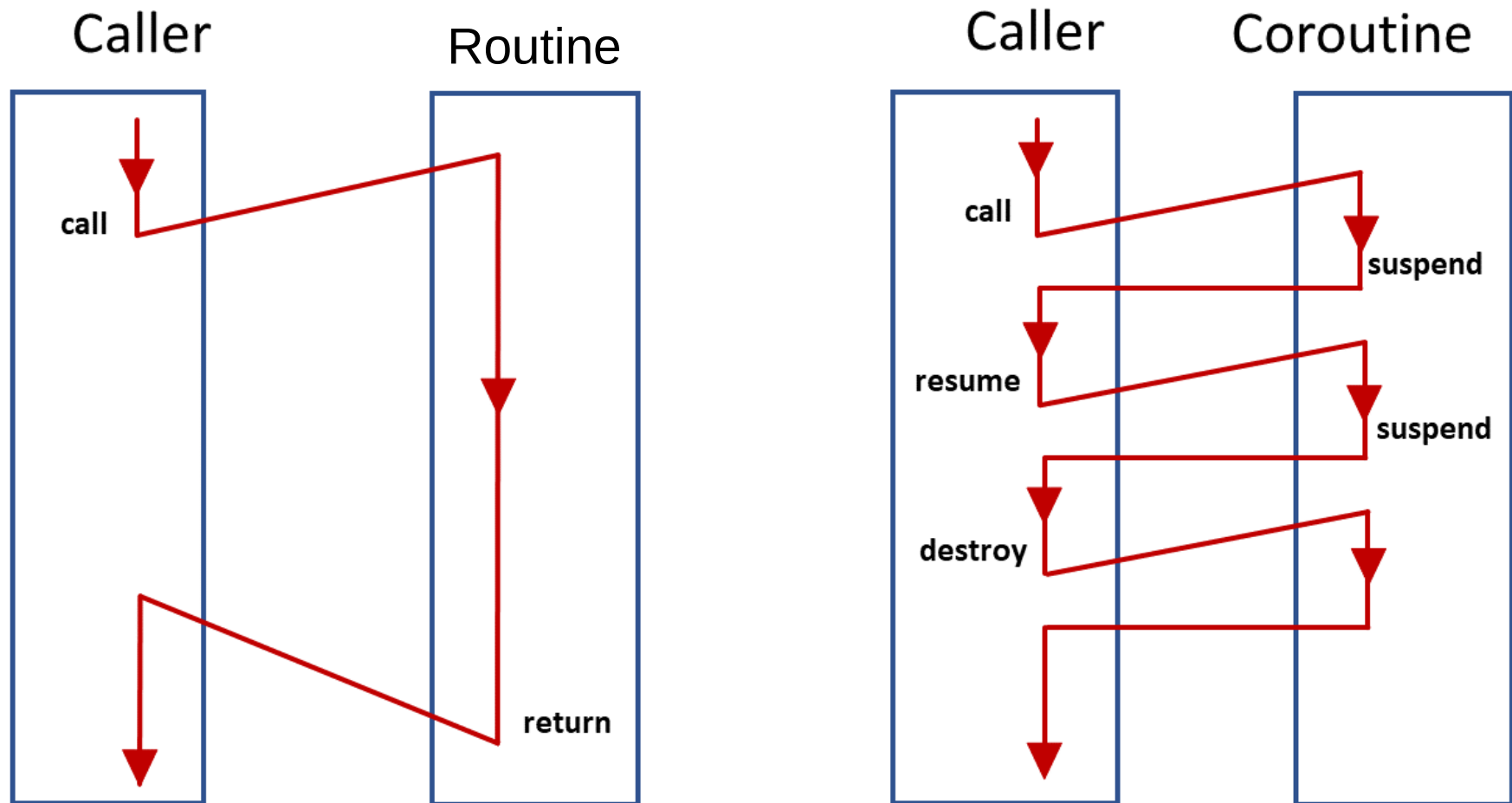
Νήματα Επιπέδου Χρήστη (User Level Threads).

- Η δρομολόγηση των νημάτων γίνεται από το σύστημα χρόνου εκτέλεσης της γλώσσας.
- Το λειτουργικό σύστημα βλέπει μόνο μια διεργασία.
- Δεν μπορούν να εκμεταλλευτούν το υλικό (πολλαπλούς πυρήνες).
- Το λειτουργικό σύστημα δεν ξεχωρίζει κλήσεις συστήματος ανά νήμα, επομένως αναστέλλει τη διεργασία.
- Ονομάζονται και green threads. Πρόσφατα το ενδιαφέρον για τα user level threads έχει αυξηθεί μέσω των coroutines και των hybrid threads.

Συνεργατική δρομολόγηση και Coroutines 1

- Coroutines: ρουτίνες που επιτρέπουν διακοπή εκτέλεσης χωρίς τερματισμό με επιστροφή (return) αλλά με ένα είδος yield() ή suspend(). Μια ρουτίνα παραχωρεί την εκτέλεση σε άλλη ρουτίνα. Όταν η αρχική ρουτίνα ξαναεκτελεστεί συνεχίζει από εκεί που διέκοψε, όχι από την αρχή.
- Η δρομολόγησή τους είναι συνεργατική (cooperative) όχι προεκτοπιστική (pre-emptive): η μία παραχωρεί την θέση της σε άλλη, μπορεί μάλιστα να ορίσει ποια ρουτίνα να συνεχίσει.

Coroutines



Suspend: προσωρινό Return, με διατήρηση του χώρου εργασίας της Coroutine. Έτσι έχουμε εναλλαγή εκτελούμενων ρουτινών, όμως πάντα μόνο μια εκτελείται (μια ροή εκτέλεσης).

Συνεργατική δρομολόγηση και Coroutines 2

- Οι coroutines μπορεί να έχουν ξεχωριστές στοίβες εκτέλεσης η κάθε μια –όπως τα νήματα – αλλά μοιράζονται ένα κοινό πλαίσιο εκτέλεσης (context), αυτό της διεργασίας, επομένως οι εναλλαγή τους είναι ταχύτερη.
- Παρέχουν ταυτοχρονισμό αλλά όχι παραλληλισμό: πάντα εκτελείται μια coroutine.
- Δεν μπορούν να εκμεταλλευτούν την ύπαρξη πολλών επεξεργαστών αλλά χρησιμοποιούνται πολύ στην διαχείριση πολλαπλών λειτουργιών κυρίως I/O.

Javascript

- Η Javascript αρχικά σχεδιάστηκε ως διεργασία με μια ροή ελέγχου, χωρίς νήματα (event loop).
- Η εξυπηρέτηση του I/O (χρήστης, δίκτυο κλπ) γίνεται με τη λογική της πολυπλεξίας, παρόμοια με την εξυπηρέτηση διακοπών ή την εκτέλεση coroutines. Το σύστημα χρόνου εκτέλεσης ελέγχει συνεχώς τη κατάσταση των διαφόρων συμβάντων (events) και εξυπηρετεί όποιο ενεργοποιηθεί.
- Σχετικά πρόσφατα η Javascript επεκτάθηκε σε χρήση νημάτων (Webworkers, Worker Threads) αλλά ακόμη η υλοποίησή τους δεν είναι τυποποιημένη, εξαρτάται από το runtime system.
- Το Node.js (θα) υποστηρίζει green threads.

Δρομολόγηση N:N

Νήματα Επιπέδου Λειτουργικού Συστήματος (Kernel Level / Platform Threads) 1

- Η δρομολόγηση των νημάτων γίνεται από το λειτουργικό σύστημα. Για κάθε διεργασία το λειτουργικό σύστημα διατηρεί πίνακα των νημάτων της και τα διαχειρίζεται πλήρως.
- Κάθε νήμα έχει τη δική του κατάσταση, σήματα κλπ άρα δρομολογείται ξεχωριστά. Η διεργασία εκτελείται όσο υπάρχουν runnable νήματα.
- Σε συστήματα μοιραζόμενης μνήμης, όταν δρομολογείται μια διεργασία με πολλά runnable νήματα, καταλαμβάνει όσο το δυνατό περισσότερους διαθέσιμους επεξεργαστές.
- Λέγονται και native threads.

Δρομολόγηση N:N

Νήματα Επιπέδου Λειτουργικού Συστήματος (Kernel Level / Platform Threads) 2

- Ο συνηθέστερος τύπος νημάτων σήμερα.

Παραδείγματα:

- POSIX threads ή Pthreads
https://en.wikipedia.org/wiki/POSIX_Threads
- C++ Threads, Java Threads, Python Threads κλπ
βασίζονται στα Pthreads, αλλά τα κάνουν πιο
εύχρηστα.
- Προσοχή στη διαφορά των νημάτων του kernel (των
λειτουργικών συστημάτων) και νημάτων επιπέδου
λειτουργικού συστήματος-native threads (N:N
απεικόνιση αλλά με λιγότερα προνόμια).

Νήματα Python

- Νήματα Επιπέδου Λειτουργικού Συστήματος.
- Όλα τα νήματα είναι ορατά στο λειτουργικό σύστημα, αλλά το σύστημα χρόνου εκτέλεσης της Python διατηρεί ένα καθολικό κλείδωμα, το Καθολικό Κλείδωμα του Διερμηνευτή (Global Interpreter Lock).
- Το GIL καταλαμβάνεται από ένα νήμα κάθε φορά για συγκεκριμένο χρονομερίδιο ή μέχρι να ανασταλεί για κάποιο λόγο. Κάθε στιγμή εκτελείται μόνο ένα νήμα της Python (ισχύει στο threading module).
- Λύσεις: παραλληλισμός διεργασιών (χρήση multiprocessing module) ή
- Python 3.13: <https://peps.python.org/pep-0703/>

Δρομολόγηση N:M

Υβριδικά Νήματα (Hybrid Threads) 1

- Η δρομολόγηση των νημάτων γίνεται σε δύο επίπεδα.
- Το σύστημα χρόνου εκτέλεσης της γλώσσας διχειρίζεται N υβριδικά νήματα, αλλά το λειτουργικό σύστημα διαχειρίζεται μόνο $M < N$ νήματα επιπέδου λειτουργικού συστήματος.
- Το σύστημα χρόνου εκτέλεσης περνά στο λειτουργικό σύστημα μόνο M υβριδικά νήματα κάθε φορά προς εκτέλεση με βάση δικούς του κανόνες δρομολόγησης.

Δρομολόγηση N:M

Υβριδικά Νήματα (Hybrid Threads) 2

- Τα M νήματα επιπέδου λειτουργικού συστήματος λειτουργούν ως “δοχεία” που εκτελούν κάθε φορά M υβριδικά νήματα από τα N που διαθέτει η εφαρμογή.
- Επομένως τα N υβριδικά νήματα “μοιράζονται” πόρους των M νημάτων επιπέδου λειτουργικού συστήματος, σαν coroutines μέσα σε νήματα.
- Η τεχνική αυτή είναι χρήσιμη για τη δημιουργία και διαχείριση πολύ μεγάλου αριθμού νημάτων, όταν έχουμε πολλά I/O bound νήματα γιατί αναστέλλονται συχνά. Δεν είναι ιδιαίτερα χρήσιμη σε CPU bound νήματα που εκτελούνται συνεχώς.

Παραδείγματα για Υβριδικά Νήματα

- Win32 Threading API
- Go goroutines
<https://www.geeksforgeeks.org/golang-goroutine-vs-thread/>
- Java virtual threads (OpenJDK 19)
<https://openjdk.org/jeps/436>
- Kotlin coroutines, Erlang processes, Haskell threads
- Λέγονται και managed threads, fibers.
Πολλές διαφορετικές προσεγγίσεις

Επικοινωνία μεταξύ διεργασιών ή νημάτων: Χώροι διευθύνσεων και φυσική μνήμη

- Κάθε διεργασία έχει το δικό της χώρο διευθύνσεων.
- Ένας χώρος διευθύνσεων δεν μπορεί εύκολα να επεκταθεί σε διαφορετικές φυσικές μνήμες. Μια διεργασία εκτελείται σε μια φυσική μνήμη.
- Πολλά νήματα που αποτελούν μέρος μιας διεργασίας, εκτελούνται σε έναν χώρο διευθύνσεων και σε μια φυσική μνήμη.
- Πολλές διεργασίες μπορούν να συνυπάρχουν σε διαφορετικές φυσικές μνήμες με ξένους μεταξύ τους χώρους διευθύνσεων.
- Πολλές διεργασίες μπορούν να συνυπάρχουν σε μια φυσική μνήμη και να έχουν ξένους μεταξύ τους χώρους διευθύνσεων ή να έχουν μοιραζόμενες μεταβλητές.

Επικοινωνία μεταξύ διεργασιών ή νημάτων:

Μέθοδοι

- Σήματα ή διακοπές (interrupts): όταν βρίσκονται στην ίδια φυσική μνήμη, μέσω λειτουργικού συστήματος ή runtime system.
- Μοιραζόμενες μεταβλητές: οι χώροι διευθύνσεων των διεργασιών ή νημάτων έχουν κοινές θέσεις φυσικής μνήμης.
- Μεταβίβαση μηνυμάτων: οι χώροι διευθύνσεων των διεργασιών ή νημάτων δεν έχουν κοινές περιοχές, αλλά επικοινωνούν μέσω μοιραζόμενων ροών μηνυμάτων (μέσω δικτύου ή λειτουργικού συστήματος ή runtime system) .
- Μοιραζόμενα αρχεία: διεργασίες ή νήματα γράφουν και διαβάζουν από κοινόχρηστα αρχεία ή διοχετεύσεις.

Μοιραζόμενες μεταβλητές

- Εφαρμόζεται σε υπολογιστικό σύστημα μοιραζόμενης μνήμης.
- Εύκολος προγραμματισμός στα νήματα επειδή έχουμε κοινό χώρο διευθύνσεων: όλα τα νήματα ανήκουν στην ίδια διεργασία.
- Κάπως δυσκολότερος προγραμματισμός σε διεργασίες: εκτελούνται σε ξεχωριστούς χώρους διευθύνσεων αλλά σε μοιραζόμενη φυσική μνήμη. Χρήση ειδικών βιβλιοθηκών (πχ mmap).
- Η εφαρμογή σε υπολογιστικά συστήματα κατανεμημένης μνήμης είναι περίπλοκη, δεν χρησιμοποιείται συχνά.

Μεταβίβαση μηνυμάτων

- Εφαρμόζεται σε υπολογιστικά συστήματα τόσο μοιραζόμενης όσο και κατανεμημένης μνήμης. Και στις δύο περιπτώσεις θεωρούμε ότι έχουμε ξεχωριστούς (ιδιωτικούς) χώρους διευθύνσεων.
- Στα συστήματα μοιραζόμενης μνήμης η μεταβίβαση μηνυμάτων μπορεί να υλοποιηθεί με ουρές μηνυμάτων ή ειδικούς τύπους δεδομένων (πχ κανάλια) ή με ειδικές βιβλιοθήκες. Εφαρμόζεται σε νήματα και διεργασίες.
- Στα συστήματα κατανεμημένης μνήμης η μεταβίβαση μηνυμάτων μπορεί να υλοποιηθεί με δικτυακή επικοινωνία (πχ υποδοχές ή κλήση απομακρυσμένης διαδικασίας ή ειδικές βιβλιοθήκες). Εφαρμόζεται σε διεργασίες.

Φυσικός και λογικός διαμοιρασμός 2

Χώροι διευθύνσεων Φυσική μνήμη	Με κοινές περιοχές	Χωρίς κοινές περιοχές
Συστήματα μοιραζόμενης μνήμης	Επικοινωνία με μοιραζόμενες μεταβλητές (νήματα, διεργασίες)	Επικοινωνία με μεταβίβαση μηνυμάτων στο ίδιο σύστημα (νήματα, διεργασίες)
Συστήματα κατανεμημένης μνήμης	Σχετικά δύσκολη υλοποίηση (υπάρχει αλλά δεν συνηθίζεται)	Επικοινωνία με μεταβίβαση μηνυμάτων μέσω δικτύου (διεργασίες)

Πρότυπα (patterns) δημιουργίας νημάτων: Παράλληλος υπολογισμός

- Συνήθως έχουμε πολλά νήματα που εκτελούν παρόμοιο κώδικα αλλά σε διαφορετικά δεδομένα.

Η δημιουργία νημάτων γίνεται

- είτε με for loop: δημιουργία N νημάτων
- είτε με αναδρομικές κλήσεις: ένα δημιουργεί δύο, αυτά από δύο κλπ
- Συνήθως υπάρχει αναμονή τερματισμού των νημάτων για συλλογή των επιμέρους αποτελεσμάτων των νημάτων από το κυρίως πρόγραμμα.
- Δεν έχουμε σημαντικές καθυστερήσεις στην απόκριση νημάτων.

Πρότυπα (patterns) δημιουργίας νημάτων:

Ταυτόχρονος, κατανεμημένος υπολογισμός

- Συνήθως έχουμε νήματα με συγκεκριμένες λειτουργίες, πχ εκτύπωση, επικοινωνία με δίσκο, επικοινωνία με το δίκτυο κλπ.
- Επίσης έχουμε νήματα που δημιουργούνται από συμβάντα (events), όπως νέα σύνδεση στο δίκτυο, νέο αίτημα σε διακομιστή, νέο παράθυρο από επιλογή χρήστη.
- Τα νήματα της πρώτης κατηγορίας εκτελούνται σε όλη τη διάρκεια της εφαρμογής, μερικές φορές στο παρασκήνιο.
- Τα νήματα της δεύτερης κατηγορίας τερματίζονται με συμβάντα, όπως ολοκλήρωση I/O, ολοκλήρωση μεταφοράς δεδομένων μέσω δικτύου, κλείσιμο παραθύρου.
- Μπορεί να έχουμε απροσδιόριστες καθυστερήσεις στην απόκριση των νημάτων ή διεργασιών (λόγω I/O, δικτύου)



Παράλληλος και Κατανεμημένος Υπολογισμός I

Νήματα στη Java 1

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2025

Εικονική μηχανή και σύστημα εκτέλεσης Java

- Αν δεν έχετε ήδη, εγκαταστήστε Java Development Kit (JDK) στον υπολογιστή σας.
- IDE/Editor προαιρετικό, όποιο θέλετε.
- Αν θέλετε ρυθμίστε τη Java από τη γραμμή διαταγών (CMD) στα Windows. Δείτε για παράδειγμα το σύνδεσμο

<https://www.javatpoint.com/how-to-run-a-java-program-in-windows-10>

Μεταγλώττιση και εκτέλεση Java από τη γραμμή διαταγών 1

- Τα προγράμματα Java (*.java) μεταγλωττίζονται σε bytecode (*.class) μέ την εντολή

`$javac myprogram.java`

- Παράγεται το αρχείο

`myprogram.class`

καθώς και άλλες κλάσεις που πιθανώς περιέχονται στο αρχείο του πηγαίου κώδικα

- Το σύστημα χρόνου εκτέλεσης της Java (JRE) εκτελεί το bytecode μιας κλάσης στη γλώσσα μηχανής του επεξεργαστή που βλέπει το λειτουργικό σύστημα.

Μεταγλώττιση και εκτέλεση Java από τη γραμμή διαταγών 2

- Εκτέλεση με την εντολή

\$java myprogram

όπου myprogram η κλάση που περιέχει τη main()

- Ένα πρόγραμμα Java εκτελείται μέσα σε μια διεργασία JRE με όνομα java. Άρα όταν γράφουμε \$java myprogram δημιουργούμε μια διεργασία.
- Όταν οι κλάσεις που εκτελούμε βρίσκονται σε άλλους καταλόγους (cp ή classpath)

\$java -cp <διαδρομή> myprogram



Νήματα

Νήματα στη Java

- Τα νήματα ενός προγράμματος java δημιουργούνται και εκτελούνται μέσα στη διεργασία java που δημιουργήσαμε για την εκτέλεση του προγράμματος.
- Κάθε νήμα έχει τη δική του ακολουθιακή ροή εκτέλεσης μέσα στη διεργασία java.
- Τα νήματα υλοποιούνται ως στιγμιότυπα της κλάσης Thread.
- Η μέθοδος `main()` μπορεί να θεωρηθεί ως το πρώτο νήμα που δημιουργείται, αλλά όχι απαραίτητα το τελευταίο που τερματίζει.

Δημιουργία και εκτέλεση νημάτων *

Δύο βασικοί τρόποι δημιουργίας νημάτων:

- Ορισμός υποκλάσης της κλάσης Thread (extends Thread)

ή

- Ορισμός κλάσης που επεκτείνει τη διεπαφή Runnable (implements Runnable)
- Η μέθοδος run() πρέπει να οριστεί οπωσδήποτε. Είναι public void και δεν δέχεται ορίσματα.

** Η Java διαθέτει και άλλες μεθόδους δημιουργίας νημάτων, για παράδειγμα μέσω Executors, Thread Factories, Streams (Lambda expressions). Όμως η παρουσίαση πολλών μεθόδων σε αυτή τη φάση μάλλον θα δημιουργούσε σύγχυση.*

Δημιουργία και εκτέλεση νημάτων 2

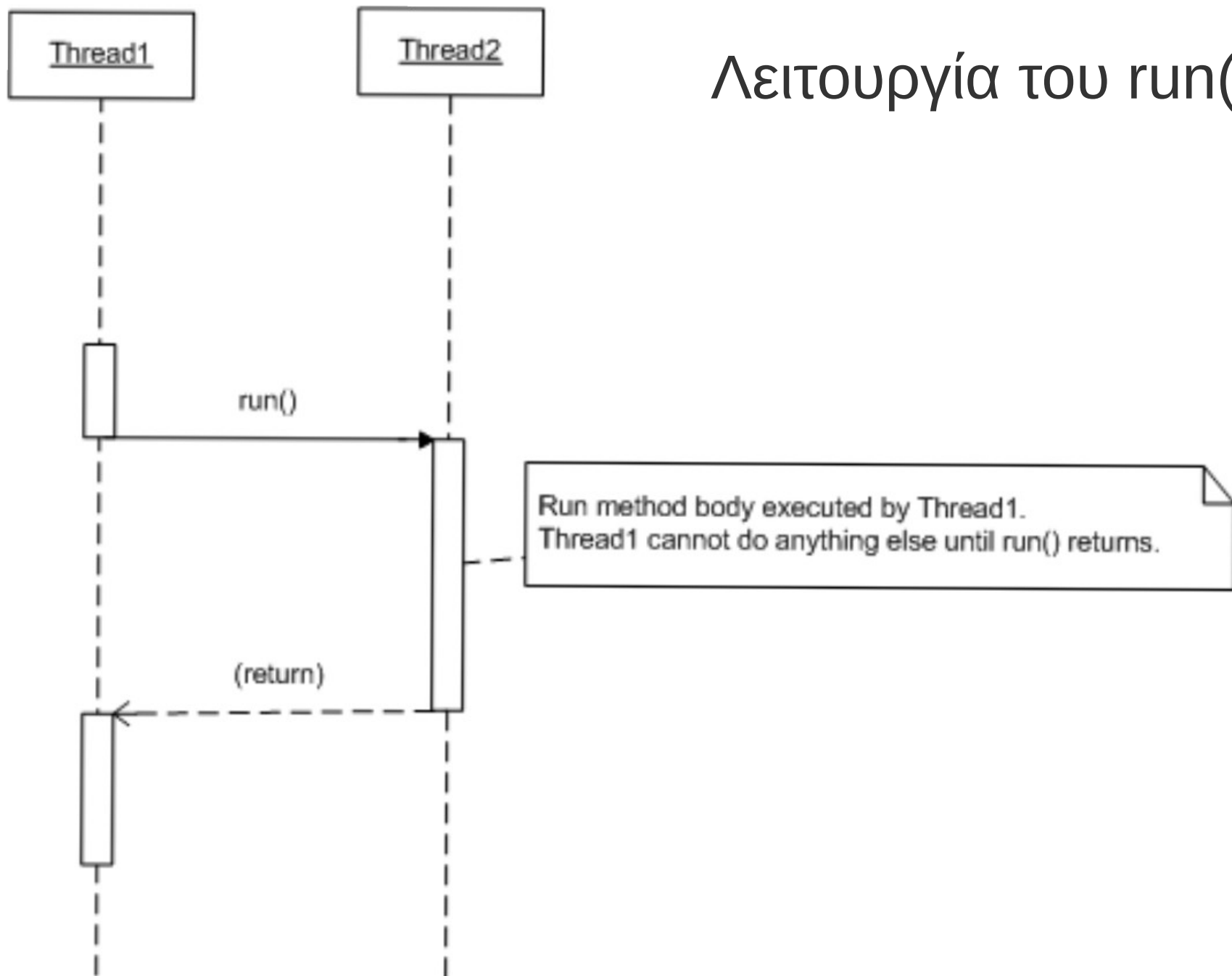
- Δημιουργία στιγμιότυπου (αντικειμένου) της κλάσης νήματος που ορίσαμε (new).
- Στη περίπτωση του Runnable: επιπλέον πέρασμα του Runnable αντικειμένου στο κατασκευαστή της κλάσης Thread.*
- Προσοχή: η δημιουργία νήματος είναι διαφορετική από την εκτέλεση νήματος
- Η βασική μέθοδος εκτέλεσης νήματος είναι με τη κλήση της μεθόδου start() -> προκαλεί εκτέλεση της αντίστοιχης run() και άμεση επιστροφή του καλούντα.

* Η χρήση Runnable είναι γενικότερη, δίνει ευελιξία, αλλά για απλές εφαρμογές δεν εμφανίζει ουσιαστική διαφοροποίηση.

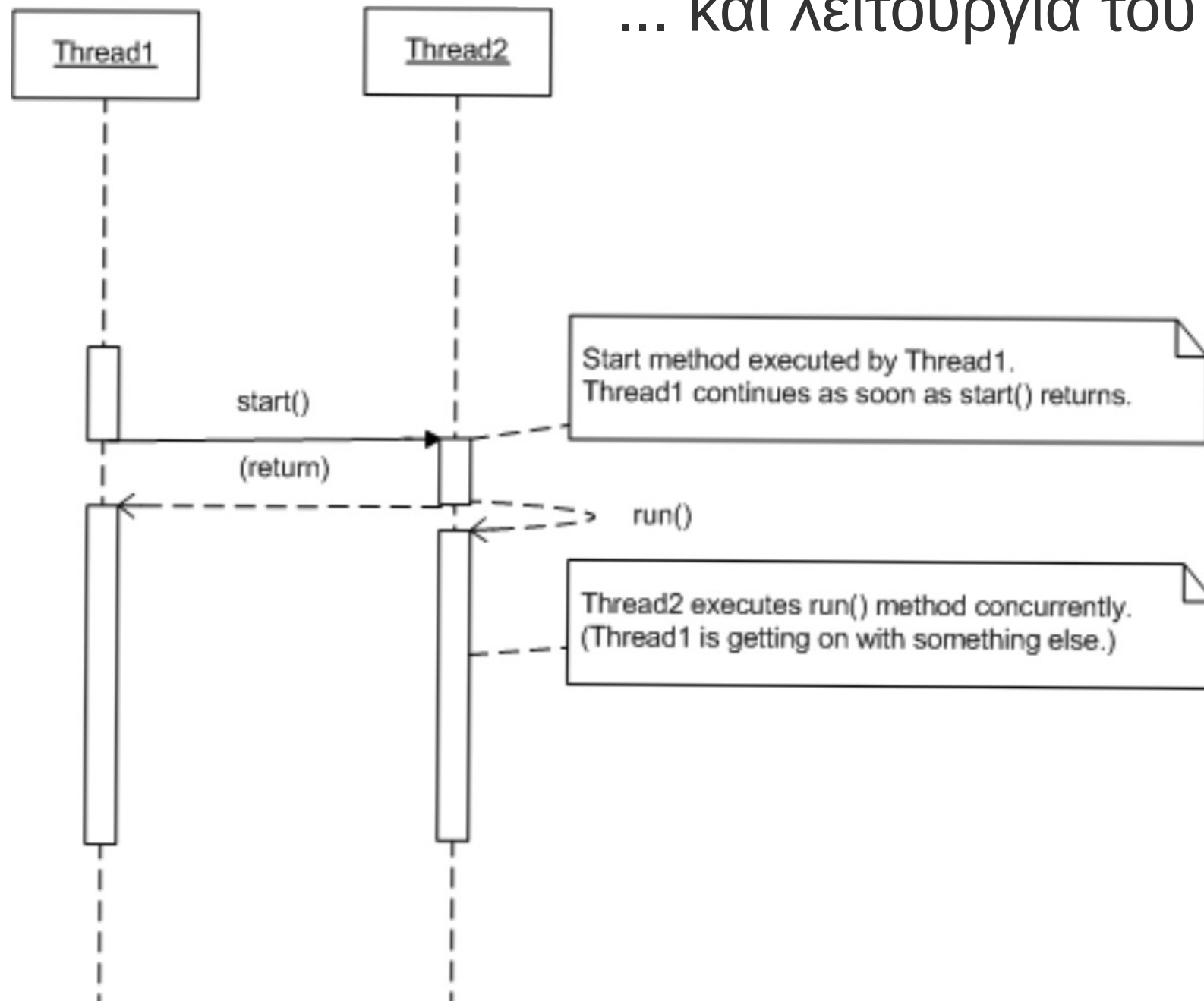
Δημιουργία και εκτέλεση νημάτων 3

- (Προαιρετικό) πέρασμα ορισμάτων από τη `main()` στα νήματα μέσω της μεθόδου κατασκευής (κατασκευαστή) κατά τη δημιουργία του νήματος.
- Οι τιμές των ορισμάτων αντιγράφονται από τη στοίβα κλήσεων της `main()` στις μεταβλητές στιγμιοτύπου του νήματος (μοιραζόμενη μνήμη).
- Η `run()` μπορεί να καλέσει άλλες μεθόδους.
- Τερματισμός του νήματος με τον τερματισμό της `run()`.

Λειτουργία του run() ...



... και λειτουργία του start()



(Κανονικός) τερματισμός νήματος

- Ένα νήμα τερματίζει όταν τελειώνει η εκτέλεση της μεθόδου `run()`.
- Μια διεργασία Java εκτελείται μέχρι το τελευταίο νήμα* να τερματίσει.
- Η `main()` μπορεί να τερματιστεί πριν το τελευταίο νήμα.
- Τα νήματα δεν είναι επανεκκινήσιμα: αν ξανακληθεί η μέθοδος `start()` στο ίδιο νήμα προκαλεί `InvalidThreadStateException`.

** Εξαιρούνται τα νήματα - δαίμονες (daemon): βοηθητικά νήματα. Αυτά τερματίζονται μαζί με το τελευταίο κανονικό νήμα.*

Αναμονή (κανονικού) τερματισμού 1

- Ένα νήμα αναστέλλει την εκτέλεσή του (Blocked) αν καλέσει τη μέθοδο `t.join()`, δηλαδή αναμένει τον τερματισμό του νήματος – στόχου `t`.
- Βασική σύνταξη `join` (αυτή η σύνταξη ισχύει γενικά όταν εκτελούμε εντολές που προκαλούν αναστολή)

```
try {  
    t.join();  
} catch (InterruptedException e) { }
```

Αναμονή (κανονικού) τερματισμού 2

- Αυτή είναι η συνήθης μέθοδος να επιβάλλουμε στο νήμα - καλούντα (πχ το `main()`) να αναστείλει την εκτέλεσή του μέχρι να τερματίσει κάποιο νήμα – στόχο.
- Η αναμονή μπορεί να περιοριστεί χρονικά `t.join(timeout)`. Αν περάσει ο χρόνος η `join` επιστρέφει από την αναστολή.
- Αν το νήμα – στόχος έχει τερματιστεί ή δεν έχει ξεκινήσει τότε η `join()` επιστρέφει αμέσως.
- Αν το νήμα καλέσει τη `join()` στον εαυτό του (!) πέφτει σε κατάσταση διαρκούς αναστολής ...

Παραδείγματα

- Τα παραδείγματα βρίσκονται στον φάκελλο `Code/HelloJavaThreads`
- `HelloAThread.java`: απλή χρήση `extends Thread`
- `HelloThread.java`: ομάδα νημάτων (`for loop`)
- `HelloThreadArgs.java`: κατασκευαστής, ορίσματα νήματος
- `HelloRunnable.java`: όπως το προηγούμενο αλλά με `implements Runnable`